

Program

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
// Maximum size of the stack
// Stack structure definition
struct Stack {
int arr[MAX];
int top;
};
// Function prototypes
void initializeStack(struct Stack *stack);
int isFull(struct Stack *stack);
int isEmpty(struct Stack *stack);
void push(struct Stack *stack, int element);
int pop(struct Stack *stack);
void displayStack(struct Stack *stack);
int main() {
struct Stack stack;
int choice, element;
initializeStack(&stack);
while (1) {
printf("\nStack Operations Menu:\n");
printf("1. Push\n");
printf("2. Pop\n");
printf("3. Display\n");
printf("4. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
switch (choice) {
case 1:
printf("Enter element to push: ");
scanf("%d", &element);
push(&stack, element);
break;
case 2:
element = pop(&stack);
if (element != -1)
printf("Popped element: %d\n", element);
break;
case 3:
displayStack(&stack);
break;
```

```

case 4:
exit(0);
default:
printf("Invalid choice! Please enter a valid option.\n");
}
}
return 0;
}
// Function to initialize the stack
void initializeStack(struct Stack *stack) {
stack->top = -1;
}
// Function to check if the stack is full
int isFull(struct Stack *stack) {
return stack->top == MAX - 1;
}
// Function to check if the stack is empty
int isEmpty(struct Stack *stack) {
return stack->top == -1;
}
// Function to push an element onto the stack
void push(struct Stack *stack, int element) {
if (isFull(stack)) {
printf("Error: Stack overflow. Cannot push element.\n");
return;
}
stack->arr[++stack->top] = element;
printf("Element pushed: %d\n", element);
}
// Function to pop an element from the stack
int pop(struct Stack *stack) {
if (isEmpty(stack)) {
printf("Error: Stack underflow. Cannot pop element.\n");
return -1;
}
return stack->arr[stack->top--];
}
// Function to display the stack elements
void displayStack(struct Stack *stack) {
if (isEmpty(stack)) {
printf("Stack is empty.\n");
return;
}
printf("Stack elements: ");

```

```
for (int i = stack->top; i >= 0; i--) {  
    printf("%d ", stack->arr[i]);  
}  
printf("\n");  
}
```